Attestation-Aware Scheduling for Verifiable AI Placement on SEV-SNP Confidential Kubernetes Nodes

Anonymous for review

***Abstract*—Kubernetes places pods using non-cryptographic, mutable cluster metadata — labels, `nodeSelector`, `RuntimeClass` — none of which carries cryptographic, freshness, or revocation semantics. For sensitive workloads on confidential hardware this is unsafe: a forged label or a race on mutable state can place a workload on a node that is not, or is no longer, attested. We present an attestation-aware Kubernetes scheduler that makes *verified* attestation evidence a first-class scheduling signal and binds the final placement decision to independently verifiable cryptographic evidence. A trust chain separates an untrusted node attester (which may only emit a raw report) from a central verifier (the sole writer of attestation evidence); the scheduler filters and scores nodes by fresh, non-revoked, verified evidence and performs a fail-closed `PreBind` re-check that removes the externally schedulable gate-removal-to-bind TOCTOU window left open by a check-then-ungate design; and an Ed25519 placement token lets a third party verify offline where a workload ran. We formalise six placement properties and evaluate on *real* AMD SEV-SNP confidential AKS hardware (Standard_DC8as_v6, westus2) with real Azure MAA guest attestation: the multi-node snapshot contains four active confidential nodes with verified real evidence, while empty/invalid tokens are never marked real. Our central result is a head-to-head comparison with a strong scheduling-gate baseline: it leaves a measured $\sim$1.1 s attestation TOCTOU window (median; Mann–Whitney $p < 10^{-5}$, Cliff's δ=1.0) that our fail-closed design removes from the externally schedulable path while additionally producing an offline-verifiable placement artifact. Layer ablations exercise admission, scheduling, and token checks. A real AKS adversarial campaign blocks all A1–A10 attack instances (300/300), including freshness, revocation, policy-race, simulated-runtime, token-tamper and forged evidence attacks; A11 separately shows fail-closed behaviour when confidential GPU attestation is requested but not available. The AKS B1–B5 latency campaign uses 30 measured runs per baseline; the comparable client-observed median is 1167.0 ms for B5, while B5's scheduler-internal phase median is separately 216.232 ms. To keep the AI claim concrete, the evaluation runs three governed AI workloads (OpenAI chat, OpenAI embedding, and a local CPU model) from attestation-scheduled pods; all pass placement-token verification. We claim node-level SEV-SNP attested placement only — not pod-level attestation, confidential GPUs, Intel TDX, OpenAI service confidentiality, or model confidentiality.**

I. Introduction

Organisations increasingly run sensitive AI inference on shared Kubernetes clusters while wanting hardware-backed guarantees that a workload executes only on a genuinely confidential node. Confidential VMs (AMD SEV-SNP, Intel TDX) and their attestation services now make it possible to *prove* that a node is a real trusted execution environment [1], [2]. Yet Kubernetes places pods using non-cryptographic, mutable cluster metadata: labels, `nodeSelector`, and `RuntimeClass` carry no cryptographic backing and no freshness or revocation semantics. An attacker who can set a label or win a race on mutable cluster state can therefore cause a sensitive workload to be scheduled onto a node that is not (or is no longer) attested.

We ask: *how can verified attestation evidence become a first-class signal in the Kubernetes scheduling cycle, and how can the resulting placement decision be made independently verifiable?* We answer with an attestation-aware scheduler for confidential AI inference, evaluated on **real AMD SEV-SNP confidential AKS nodes**. We deliberately scope claims to *node-level* attestation: on confidential-VM node pools, the evidence attests the confidential worker node/VM, not a per-pod TEE. We make this explicit throughout and do not claim pod-level attestation, confidential GPUs, or Intel TDX.

a) Contributions.:

- A trust chain that separates the untrusted node *attester* (a node-resident agent that may only emit a `RawAttestationReport`) from a central *verifier* (the sole writer of `AttestationEvidence`), mirroring the IETF RATS roles [3]. A compromised node cannot forge evidence (attack A10).
- An attestation-aware scheduler that filters, scores and binds pods only to nodes with fresh, non-revoked, verified evidence, with a *fail-closed* `PreBind` re-check that removes the externally schedulable gate-removal-to-bind Time-Of-Check-to-Time-Of-Use window, and an Ed25519 *placement token* that an independent `verify-placement` tool validates offline.
- Real Azure MAA / SEV-SNP guest attestation: on `Standard_DC8as_v6` AKS confidential hardware, the verifier produced `evidenceMode=real` for active confidential nodes with distinct, RS256-verified MAA tokens — and, by construction, refuses to mark empty/invalid tokens real. A multi-node snapshot used in the evaluation contains four active confidential nodes with real verified evidence.
- A formalisation of six placement properties (P1–P6, §VI) and an argument that the architecture enforces them independently of the specific TEE, plus a residual-window analysis of the scheduling cycle that identifies PreBind as the sole remaining check-to-bind gap.
- An evaluation on real hardware whose *central* result is

a head-to-head comparison with a strong scheduling-gate baseline (B4): B4 leaves a measured $\sim$1.1 s gate-removal-to-bind TOCTOU window (Mann–Whitney $p < 10^{-5}$, Cliff's δ=1.0) that our fail-closed design removes from the externally schedulable path while adding an offline-verifiable placement artifact. The deployed AKS adversarial campaign blocks all A1–A10 instances (300/300), and A11 separately confirms fail-closed behaviour for GPU-required workloads without GPU attestation. Layer ablations show why admission, scheduler, and token checks are needed. The AKS B1–B5 latency campaign uses 30 measured runs per baseline; B5's comparable client-observed median is 1167.0 ms, and its scheduler-internal phase median is separately 216.232 ms. Three governed AI workloads (OpenAI chat, OpenAI embedding, and a local CPU model) run from attestation-scheduled pods and all pass placement-token verification.

This work does not replace pod-level confidential-container attestation systems; it makes verified node attestation a scheduling-time signal and binds the final placement to independently verifiable cryptographic evidence. Confidential AI inference is the motivating use case and is represented by minimal real AI workloads in our AKS evaluation; confidential GPU validation is left to a future NVIDIA H100 phase.

II. Background

a) Confidential VMs and SEV-SNP.: A Confidential VM encrypts and integrity-protects an entire guest so that the host, hypervisor and cloud operator are outside its trust boundary. AMD SEV-SNP adds strong memory integrity and a hardware-rooted attestation report; Azure exposes this via Guest Attestation and the Microsoft Azure Attestation (MAA) service, which returns a signed JWT asserting, among other claims, `x-ms-attestation-type=sevsnpvm` [1], [4]. Remote attestation lets a relying party verify such evidence; the IETF RATS architecture standardises the Attester/Verifier/Relying-Party roles and insists that appraisal be independent of evidence production [3], [5].

b) Kubernetes scheduling and admission.: The default scheduler binds pods to nodes using the scheduling framework (PreFilter/Filter/Score/Reserve/Permit/Bind); a cluster may also run additional schedulers for pods that request them by name. Placement inputs — labels, `nodeSelector`, node affinity, `RuntimeClass` — are ordinary, attacker-writable cluster objects with no cryptographic backing. Admission webhooks and `schedulingGates` can gate a pod until an external controller approves it, but the gate check and the eventual bind are separate events, so a check-then-ungate design has an inherent TOCTOU gap. `RuntimeClass` selects a runtime handler (e.g. a confidential-container runtime) but carries no freshness, revocation or node-identity semantics of its own.

c) Confidential containers vs. confidential nodes.: Confidential Containers (CoCo) with Trustee run each pod inside its own TEE and gate secret release on pod-level attestation; recent work demonstrates pod-level remote attestation on Kubernetes [6], [7], [8]. On confidential-VM node pools, by contrast, the TEE is the *node*: evidence attests the confidential worker VM, not each pod. Our work targets the latter and is explicit about it; it is complementary to pod-level systems.

TABLE I
POSITIONING VS. THE CLOSEST CONFIDENTIAL-KUBERNETES SYSTEMS. *Sched.* = ATTESTATION IS CONSULTED AT SCHEDULING TIME; *TOCTOU* = REMOVES THE EXTERNALLY SCHEDULABLE CHECK-TO-BIND WINDOW; *Verif. artifact* = EMITS AN OFFLINE-VERIFIABLE PLACEMENT PROOF; *Freshness* = ENFORCES EVIDENCE AGE/REVOCATION AT BIND.

System	Level	Sched.	TOCTOU	Verif. artifact	Freshn
CoCo/Trustee [7]	pod	—	—	—	runtir
dstack [6]	pod	—	—	—	runtir
Constellation	cluster	—	—	—	boo
C8s (conf. control plane)	cluster	—	—	—	—
`schedulingGate`+ctrl. (B4)	node	✓	partial	—	✓
This work (B5)	node	✓	✓	✓	✓

III. Related Work

We position our contribution against six strands of prior work: (i) TEE hardware and confidential VMs, (ii) remote attestation architecture and protocols, (iii) TEE systems and secure containers, (iv) confidential containers and confidential Kubernetes, (v) confidential ML/AI inference, and (vi) GPU trusted execution. Throughout, we make explicit what we do *not* claim: our evaluation targets AMD SEV-SNP *node-level* attestation on Confidential VM node pools, not per-pod TEE attestation and not confidential GPUs.

A. TEE hardware and confidential VMs

Confidential Virtual Machines shift the trust boundary from a process-scoped enclave to a whole encrypted VM. AMD SEV-SNP and Intel TDX are the dominant CVM substrates; their architectures, interfaces and security properties differ in subtle ways that complicate reasoning about evidence semantics [1], [2]. Formal analysis of the SEV-SNP software interface has proved most secrecy, authentication, attestation and freshness properties in a symbolic model while flagging residual weaknesses [4], and micro-architectural side channels against SEV (e.g. the ciphertext side channel [9]) motivate treating a confidential node's evidence as necessary but bounded — reinforcing our freshness and revocation checks rather than assuming a node is trusted forever. Foundational enclave designs (Intel SGX [10], the open RISC-V Keystone framework [11]) and Arm's confidential-compute direction [12], [13] establish the primitives our evidence is ultimately rooted in. Recent systematizations argue that a missing *abstraction layer* hinders broad CVM adoption [14]; our work contributes one such layer at the orchestration tier: verified attestation evidence as a first-class scheduling signal. Crucially, none of these works addresses *where* in a cluster a sensitive workload is placed, nor binds a placement decision to the evidence.

B. Remote attestation architecture and protocols

The IETF RATS architecture [3] standardises the Attester/Verifier/Relying-Party roles and the evidence→attestation-result flow. A recurring, security-critical requirement is the strict separation between the party that *produces* evidence and the party that *appraises* it; open attestation systems such as OPERA [5] and RA-TLS-style trusted channels [15], [16] operationalise this separation for enclaves, and in-enclave verification of policy compliance has been explored [17]. Bridging attestation technologies (e.g., attesting pre-SNP SEV VMs via SGX enclaves) further underlines that appraisal must be an independent, trusted step [18]. Our design embodies the RATS separation *inside Kubernetes*: a node-resident agent may only emit an untrusted `RawAttestationReport`, while a single central verifier is the sole writer of the appraised `AttestationEvidence`. This directly answers the reviewer questions "who verifies attestation?" and "can a node self-attest?" — it cannot.

C. TEE systems and secure containers

Library-OS and secure-container systems (SCONE [19], Graphene/Gramine [20], Occlum [21]) run unmodified or lightly modified workloads inside enclaves, and language- and storage-level confidential runtimes extend the model [22], [23]. These systems harden the *inside* of a confidential workload. They are orthogonal and complementary to our work, which does not modify the runtime: we govern *admission and placement* so that a sensitive pod only ever binds to a node whose attestation evidence has been independently verified, fresh, and non-revoked.

D. Confidential containers and confidential Kubernetes

The closest ecosystem is Confidential Containers (CoCo) with the Trustee attestation/secret-release service [7], and the broader "confidential Kubernetes" pattern of running nodes or pods inside CVMs [8]. Pod-level remote attestation for Kubernetes workloads has recently been demonstrated on dstack [6], and empirical work has characterised trust-boundary vulnerabilities in TEE containers [24]. We deliberately do *not* claim pod-level attestation: on AKS SEV-SNP Confidential VM node pools the evidence refers to the confidential worker node/VM, so we report *attested-node placement*. Our contribution is orthogonal to CoCo/Trustee: rather than gating secret *release* at runtime, we make verified evidence a *scheduling-time* signal and bind the final `AIPlacementDecision` to independently verifiable cryptographic evidence via a signed placement token. Compared with industrial confidential-Kubernetes distributions (e.g. Constellation-style whole-cluster CVMs, and C8s-style confidential control planes), which secure the cluster substrate, we target scheduling-time enforcement and a verifiable placement artifact; the two are composable.

a) Why not simpler mechanisms?: A Q1 reviewer will ask why node labels, `RuntimeClass`, or `schedulingGates` plus an external controller do not already suffice. Labels and `nodeSelector` are attacker-forgeable cluster metadata with no cryptographic backing; `RuntimeClass` selects a runtime handler but carries no freshness, revocation, or node-identity semantics; and a `schedulingGates`+controller design (our strong baseline B4) can check evidence *before* ungating but leaves a Time-Of-Check-to-Time-Of-Use (TOCTOU) window between the check and the bind, and produces no offline-verifiable placement artifact. We quantify this TOCTOU window against our fail-closed `PreBind` re-check and show that only our design emits a signed token that an independent `verify-placement` tool validates without cluster access.

E. Confidential ML/AI inference

A large body of work protects models and inference inside TEEs, via DNN partitioning between trusted and untrusted compute (Slalom [25], DarkneTZ [26], ShadowNet [27], TEESlice [28]), secure accelerators [29], [30], and secure serving systems [31], [32], [33], [34], [35], [36], [37]. Systematizations map the design space and its pitfalls [38], [39], and TEE-shielded partitioning has itself been shown insecure under strong adversaries [40]. These works answer *how* to run a confidential model; they assume the workload already runs on a trustworthy node. Our work answers the prior, orchestration-level question — *how to guarantee, verifiably, that it was placed on an attested node* — and is therefore complementary to all of them.

F. GPU trusted execution (future work)

GPU TEEs (Graviton [41], Telekine [42]) and production confidential GPUs (NVIDIA Hopper H100, analysed for performance [43] and security [44]) extend confidential execution to accelerators. We do not evaluate confidential GPUs: the DCasv6 quota used here is CPU-only SEV-SNP. GPU-aware placement is expressed in our policy schema and left as validated future work with NCC H100 hardware.

G. Orchestration integrity and verifiable provenance

Finally, supply-chain and provenance work — hardware-attested ML property cards [45], and hardened Kubernetes deployments [46], [47] — shares our goal of making security claims *verifiable* rather than asserted. Our signed `AIPlacementDecision` extends this ethos to the placement decision itself: the "where did my confidential workload run?" question becomes answerable offline by a third party.

IV. Threat Model and Trust Assumptions

A. System and trust boundary

We consider a multi-tenant Kubernetes cluster in which some worker nodes are AMD SEV-SNP Confidential VMs (CVMs). A sensitive AI-inference workload must run only on a node whose attestation evidence has been *independently verified*, is *fresh*, and is *not revoked*. The trust chain has three roles that mirror the IETF RATS architecture [3]: a node-resident *attester* (the `node-attestation-agent`), a central *verifier* (the `central-verifier`, the sole writer of `AttestationEvidence`), and *relying parties* (the attestation-aware scheduler and the `verify-placement` tool).

a) Trusted computing base (TCB).: We trust: the Kubernetes API server and etcd; the AMD SEV-SNP hardware root of trust and the Microsoft Azure Attestation (MAA) verifier within the standard SEV-SNP threat model; the central-verifier's appraisal logic; and the scheduler's Ed25519 signing key. The confidentiality/integrity guarantees of the SEV-SNP CVM itself are assumed per its specification and are supported by formal analysis of the SEV-SNP software interface [4].

b) Explicitly out of scope.: A malicious or compromised Kubernetes API server or etcd; a cloud provider that violates the SEV-SNP hardware guarantees; micro-architectural side channels [9]; physical attacks; and compromise of the scheduler's private signing key. These are orthogonal, hardware- or platform-level threats addressed by prior work and by the SEV-SNP TCB, not by an orchestration-layer defense.

B. Adversary classes

We define four adversaries of increasing capability.

- ADV-1 — Can create/modify pods and application-layer objects within a namespace it controls, and set arbitrary pod labels, annotations, `nodeSelector`, and `runtimeClassName`. It cannot forge attestation evidence.
- ADV-2 — Controls a provisioned node identity and its node-agent ServiceAccount. It may attempt to submit attestation material for itself or to write evidence directly.
- ADV-3 — Controls a genuine CVM and its node-agent (post-attestation). It may try to reuse or replay evidence, or to keep serving after its evidence is revoked or expires.
- ADV-4 — Can observe cluster state and modify a matched `ConfidentialInferencePolicy`, flip a label, revoke evidence, or tamper with a placement token *between* the scheduler's Filter and Bind phases (a TOCTOU adversary). It cannot forge valid Ed25519 signatures.

C. Security properties

The design targets six properties, each falsifiable by an attack: *Placement Safety* (a sensitive pod binds only to a node with valid evidence), *Evidence Freshness* (stale/revoked evidence is rejected, even if it becomes stale mid-scheduling), *Identity Binding* (the decision is bound to the exact pod, node, policy and evidence), *Label Resistance* (attacker-forgeable metadata cannot grant placement), *Runtime Consistency* (the effective runtime matches policy, and simulated runtimes are refused in production), and *Verifiable Placement* (the decision carries an offline-verifiable cryptographic token).

D. Attack catalog

Table II (source: `article1/paper/tables/adversary_attack_mapping.csv`) enumerates attacks A1–A10, the adversary that mounts each, the targeted property, the expected defense, and the raw-result artifact that evidences the outcome. The principal security evaluation uses the real AKS SEV-SNP campaign. Local `kind` runs are CI/regression artifacts only and are not counted as paper security results. Attacks not yet re-run on AKS are identified as unit/regression coverage rather than real-cluster evidence. Crucially, A10 confirms that a node-agent principal *cannot* write `AttestationEvidence`: a forged `RawAttestationReport` is appraised by the central verifier to `evidenceMode=unverified`, `verificationStatus=Failed`, and never yields real evidence.

TABLE II
ADVERSARY/ATTACK MAPPING (DATA: `TABLES/ADVERSARY_ATTACK_MAPPING.CSV`). PROP. = TARGETED SECURITY PROPERTY.

Attack	Adv.	Property / defense
A1	ADV-4	Label Resistance: forged label, no evidence → Pending
A2	ADV-1	Evidence Freshness: stale evidence filtered
A3	ADV-1	Placement Safety: revoked evidence filtered
A4	ADV-1	Placement Safety: wrong-TEE evidence filtered
A5	ADV-1	Runtime Consistency: non-conf. runtime denied
A6	ADV-4	Runtime Consistency: Pre-Bind policy-hash re-check
A7	ADV-3	Evidence Freshness: PreBind revocation re-check
A8	ADV-4	Verifiable Placement: tampered token rejected
A9	ADV-1	Runtime Consistency: simulated runtime denied in prod
A10	ADV-2/3	Placement Safety + Identity Binding: forged evidence → unverified

V. DESIGN

Our design has three parts: an evidence trust chain, an attestation-aware scheduling cycle, and a verifiable placement token.

A. Evidence trust chain

Following the RATS separation of roles [3], no single compromised component can both produce and bless evidence. A `node-attestation-agent` runs on each confidential node and obtains a raw attestation token (on Azure, an MAA / Guest Attestation JWT bound to a node-chosen nonce). The agent may *only* create a `RawAttestationReport`; RBAC forbids it from writing `AttestationEvidence`. A single `central-verifier` consumes raw reports and is the *only* writer of `AttestationEvidence`. It parses the MAA JWT, fetches the issuer's JWKS, verifies the RS256 signature, checks the SEV-SNP attestation-type claim, freshness (nonce, expiry), and policy, and writes `evidenceMode=real` with the token hash and a canonical claims digest. If the signature cannot be checked (keys unreachable) it writes `unverified`/`Unavailable`; if a check fails, `Failed`. It *never* fabricates a real result.

B. Attestation-aware scheduling cycle

Sensitive pods carry `schedulerName: ai-attestation-scheduler`. The scheduler implements: *PreFilter* (load the matching `ConfidentialInferencePolicy`); *Filter* (keep only nodes whose `AttestationEvidence` is verified, fresh, non-revoked and of the required TEE); *Score* (prefer fresher, real evidence); *Reserve* (create the `AIPlacementDecision`); and a fail-closed *PreBind* re-check that re-reads the policy and evidence *immediately before* binding and refuses if the policy hash or evidence changed, if evidence became revoked or stale, or on any API read error. This removes the externally schedulable time-of-check-to-time-of-use (TOCTOU) window that a check-then-ungate design exposes to the API server between gate removal and bind (§VIII, E5). A residual in-process PreBind-to-Bind interval remains bounded to the scheduler's local path; a bounded `Permit` wait admits evidence that is about to become valid, otherwise rejects.

C. Verifiable placement token

At Reserve the scheduler mints an Ed25519 *placement token* over a canonical binding of {pod UID, pod-spec hash, node identity, runtime class, evidence hash, policy hash, image/model digest, validity window}, and stores it on the signed `AIPlacementDecision`. An independent `verify-placement` CLI checks the signature and field bindings offline with only the scheduler's public key — answering "where did my confidential workload run?" to a third party without cluster access. The signing key is held in a Kubernetes Secret the scheduler mounts; the scheduler RBAC does not grant blanket secret access (§X), so a rogue scheduler cannot mint acceptable tokens.

VI. Placement Properties and Residual-Window Analysis

To move beyond a single artifact, we state the guarantees as TEE-agnostic properties and argue the *architecture* — not the specific SEV-SNP/Azure instantiation — enforces them. Let a placement be the binding of a sensitive pod p to a node n under policy π, using evidence e, producing a signed decision d with token t.

- P1 Placement Safety: p binds to n only if there exists evidence e for n that is *verified* (appraised by the central verifier), *non-revoked*, and satisfies π's TEE requirement, at the instant of bind.
- P2 Evidence Freshness: The evidence used to bind satisfies $age(e) \leq \pi.maxAge$ at the instant of bind; evidence that expires or is revoked between selection and bind cannot be used.
- P3 Identity Binding: t is valid only for the exact tuple $\langle \mathrm{uid}(p), H(\mathrm{spec}(p)), n, H(e), H(\pi) \rangle$; any mismatch is rejected offline.
- P4 Label Resistance: No attacker-writable metadata (labels, `nodeSelector`) can cause a bind that P1 would forbid.
- P5 Runtime Consistency: The confidential runtime class equals a π-allowed class, and simulated classes are refused under production mode.
- P6 Verifiable Provenance: Given only the scheduler's public key and t, a third party decides whether d is a genuine, unmodified decision for p on n under π and e, without cluster access.

a) Why the architecture enforces them.: P1/P2 hold because *selection* (Filter/Score) and *commitment* (PreBind/Bind) both consult the central verifier's `AttestationEvidence` and the two are separated by the fail-closed PreBind re-check (§V); no path binds without a fresh, verified, non-revoked read. P3/P6 hold because t is an EUF-CMA signature over the canonical tuple: forging a valid t for a different tuple reduces to forging Ed25519. P4 holds because the scheduler consults evidence, never labels, for the safety decision (falsified in E5: with the scheduler removed, a forged label bypasses; with it, it does not). P5 is enforced at admission and re-checked at PreBind. Crucially, these arguments mention SEV-SNP only through the abstract predicate "e is verified"; swapping the TEE or attestation service changes only the verifier's appraisal, not the scheduling guarantees — the result is transferable.

b) Residual-window analysis.: The scheduling cycle has four time-of-check points where mutable state (π, e, node taints) is read: PreFilter, Filter, Score/Reserve, and PreBind. A TOCTOU adversary (ADV-4) wins if state changes after the *last* check that gates the bind and before the bind itself. We therefore make PreBind the last check and perform it *immediately* before issuing the Binding, with no awaited step in between: after PreBind re-reads π and e and mints t, the only remaining operation is the `Binding Create`. The residual window is thus the Reserve→Bind local interval (creating d and issuing the Binding), during which no external actor is awaited; unlike a `schedulingGate` design (B4), there is no gate-removal event that hands control back to the API server before the bind. E4 measures this difference: B4's window is the gate-removal→bind interval an external revocation can hit ($\sim$1.1 s median), whereas B5's is the in-process Reserve→Bind interval with the re-check inside it, empirically 0 at one-second resolution. We do not claim the residual in-process interval is provably empty — a revocation landing between the PreBind read and the Binding create is a theoretical race — but it is bounded by one API write and is not externally schedulable, which is the qualitative gap between B5 and any check-then-ungate baseline.

VII. Implementation

The prototype is built with Kubebuilder / controller-runtime (Go 1.21, controller-runtime v0.17). It defines the CRDs `ConfidentialInferencePolicy`, `AttestationEvidence`, `AIPlacementDecision` and `RawAttestationReport`. Components:

- **node-attestation-agent** (DaemonSet on confidential nodes): invokes an Azure Guest Attestation helper against the MAA endpoint (`sharedwus2.wus2.attest.azure.net` in

westus2) with a per-node nonce, obtains an MAA JWT, and writes a `RawAttestationReport`. It has create-only RBAC on raw reports and cannot touch `AttestationEvidence`.

- **central-verifier**: reconciles raw reports; the MAA verification package performs real RS256/JWKS signature checking, claim, freshness and TEE-type validation, returning `Verified`/`Failed`/`Unavailable`. It is the sole writer of `AttestationEvidence.status.evidenceMode`.
- **ai-attestation-scheduler**: an out-of-tree second scheduler that handles only pods naming it, implementing the cycle of §V and binding via the Kubernetes Binding API. Placement tokens use Ed25519 with a deterministic canonical hashing of the bound fields.
- **admission webhook**: mutates sensitive pods (scheduler name, runtime class, policy-hash / model-digest annotations, scheduling gate) and validates them (digest pinning, allowed runtime classes, and refusal of simulated runtime classes in production mode).
- **verify-placement**: a standalone CLI that verifies a placement token offline from the token and the scheduler public key.

RBAC is least-privilege per component (§X). The platform is packaged as a Helm chart; the confidential node pool is provisioned by Terraform (AKS confidential-VM node pool on `Standard_DC8as_v6`). In a local `kind` cluster, evidence is explicitly `simulated` and runtime classes are simulated mappings backed by `runc`; production mode refuses these. All code, manifests, experiment harnesses and analysis scripts are in the artifact.

VIII. Evaluation

We ask: (E1) does the system *really* verify SEV-SNP attestation on production confidential hardware? (E2) can real AI workloads run from governed pods? (E3) does scheduling behave correctly across multiple confidential nodes and invalid evidence states? (E4) does the deployed system withstand an adversarial campaign? (E5) how does it compare, quantitatively, to a strong scheduling-gate baseline — our central result? (E6) is each enforcement layer, and PreBind in particular, necessary? (E7) is the placement decision cryptographically bound and offline-verifiable? (E8) what is the B1–B5 overhead? and (E9) is the scheduler itself minimally privileged? Every number is backed by a raw artifact under `article1/results/raw/`; each result states its environment.

A. Experimental environment

Table III records the exact real environment; all figures come from these raw Azure outputs, not from memory. A local `kind` cluster is used *only* for CI, debug, and regression; no `kind` number is used as principal security or performance evidence.

On `Standard_DC8as_v6` the pods run under `runc`: the node is a confidential VM (SEV-SNP), and attestation is *node-level*. AKS does not enable pod sandboxing / Kata on this SKU (no nested virtualization), so we do not claim pod-level `kata-vm-isolation`.

TABLE III
Experimental environment (from raw Azure outputs).

Cloud provider	Microsoft Azure (AKS)
Region	`westus2`
Confidential node pool	`conf`, 4 active nodes during final AKS runs
Confidential VM SKU	`Standard_DC8as_v6` (AMD SEV-SNP CVM)
Quota family	Standard DCasv6 Family vCPUs (32)
vCPU (confidential active/max)	32 active / 32 max
System pool	1× `Standard_D2s_v3` (non-confidential)
Runtime class	`runc` (node-level SEV-SNP; not pod sandbox)
Attestation service	Microsoft Azure Attestation (MAA)
MAA endpoint	`sharedwus2.wus2.attest.azure.net`
Attestation type claim	`sevsnpvm`
Experiment date	2026-07-06

B. E1: Real SEV-SNP attestation

The node-attestation-agent collected genuine MAA tokens and the central verifier appraised them. The final multi-node snapshot (`multinode-node-selection/evidence-initial.json`) contains four active `Standard_DC8as_v6` confidential nodes, each with non-simulated `evidenceMode=real`, `verificationStatus=Verified`, a `maaTokenHash`, and a `claimsDigest`. The filtered summary (`attestation-real-summary.json`) is used to exclude old/orphan evidence objects from earlier autoscaler instances. The fact that the token passed RS256 verification against MAA's published JWKS establishes authentic hardware attestation, not a replayed constant. Critically, the same verifier returns `unverified`/`Failed` for an empty or malformed token (observed during bring-up and enforced by unit tests `TestUnavailableWhenKeysUnreachable`, `TestBadSignatureFails`): the pipeline never fabricates a real result.

C. E2: Governed AI workloads

Confidential AI inference is the motivating use case, so the AKS campaign runs real, minimal AI workloads from attestation-governed pods rather than using AI as a slogan. The workload CSV (`ai_workloads.csv`) contains three passing workloads: `openai-chat-minimal` (OpenAI Responses API, 3/3 requests, median 2216.815 ms), `openai-embedding-minimal` (OpenAI embeddings, 3/3, median 705.118 ms), and `local-cpu-minimal` (a deterministic local CPU model, 5/5, median 0.001 ms). All three pods are governed by a `ConfidentialInferencePolicy`, receive `placement_decision=allow`, and pass offline placement-token verification. We evaluate attested node placement and binding to model/image digest identifiers; we do not claim model confidentiality, OpenAI service confidentiality, GPU execution, or large-model throughput.

D. E3: Multi-node node qualification

To avoid evaluating a scheduler on a single active confidential node, we scaled the AKS confidential pool to four active `Standard_DC8as_v6` nodes under the 32-vCPU DCasv6 quota. The snapshot used by `run_multinode_node_selection.sh` records four active confidential nodes with real verified evidence. The experiment then exercises four cases (`multinode_node_selection.csv`): a valid confidential node is selected and bound with a verified placement token; a node whose real evidence is temporarily marked revoked is not bound; a node whose real evidence is temporarily aged beyond the freshness bound is not bound; and a non-confidential system node selected by `nodeSelector` but lacking `AttestationEvidence` is not bound. The revoked/expired modifications are backed up and restored by the harness before exit.

E. E4: Deployed adversarial campaign

We executed, against the deployed AKS system, attacks A1–A10, each from an adversarial principal, $N{=}30$ per attack. The merged raw matrix (`security_attacks_A1_A11.csv`) contains 300 A1–A10 attack instances and all are blocked; `security_attack_matrix.csv` reports Wilson 95% intervals per attack. The rerun includes stale evidence (A2), revoked evidence (A3), wrong TEE (A4), production-forbidden/simulated runtime use (A5/A9), missing model digest (A5b), policy and revocation races between Filter and PreBind (A6/A7), token tampering (A8), and forged raw reports / compromised node-agent attempts (A10). A11 is reported separately as a GPU-scope fail-closed check: a workload requiring confidential GPU attestation is denied when no such evidence exists, but we do not claim GPU confidential-computing evaluation.

F. E5: Strong baseline B4 vs. B5 (central result)

This is the paper's central quantitative result. **B4** is a genuinely strong, non-strawman baseline: an external controller places a `schedulingGate` on the pod, verifies the *same* real `AttestationEvidence` (TEE, freshness, revocation), removes the gate, and lets the default scheduler bind. It is exactly the design a careful engineer would build with today's Kubernetes primitives. **B5** is our attestation-aware scheduler with the fail-closed `PreBind` re-check and a signed placement token. Over $N{=}30$ runs each (`b4_vs_b5.csv`, Fig. 2), B4 exhibits a gate-removal→bind **TOCTOU window (median ≈1.1 s, up to ∼2.8 s)** — the window in which a revocation or policy change is not re-checked before bind — and produces *no* offline-verifiable artifact. B5 eliminates this externally schedulable gate-removal-to-bind window: the policy/evidence re-check runs inside `PreBind`, and B5 emits a token that `verify-placement` validates without cluster access. A residual in-process PreBind-to-Bind interval remains theoretically possible, but it is bounded to a single local scheduling path and does not hand control back to the API server between check and bind. The difference is statistically decisive (Mann–Whitney $U{=}900$, $p < 10^{-5}$; Cliff's $\delta{=}1.0$, "large"; `b4_vs_b5_stats.csv`). The message is threefold: schedulingGates are strong but leave an externally controllable check-to-bind gap; PreBind removes that gap from the schedulable path; and the placement token makes the outcome auditable — properties B4 cannot provide.

G. E6: Ablation — is each layer necessary?

We disable enforcement layers one at a time and re-run the corresponding attack (`ablation.csv`, Fig. 3). With *policy-only* (L0, no scheduler enforcement) a forged-label pod is *admitted* (A1 bypasses): labels alone are not a control. Adding admission (L1) blocks the runtime-class attack (A5). Adding the attestation scheduler (L2) blocks the forged-label attack (A1: the pod stays `Pending` with no valid evidence). Adding the placement token (L3) blocks token tampering (A8). The ablation makes the central security claim falsifiable: removing our layers restores the bypass, so the block rate is attributable to the mechanism, not to the choice of scenarios. The intra-scheduler Filter-only vs. +PreBind ablation requires a build flag and is noted as future work; E5 already isolates PreBind's effect quantitatively.

H. E7: Identity binding & verifiable placement

The placement token binds {pod UID, pod-spec hash, node identity, evidence hash, policy hash}. Using `verify-placement` offline (`identity_binding.csv`, Fig. 4), the correct binding verifies, and mutating *any* bound field — pod UID, pod-spec hash, node identity, evidence hash, or policy hash — makes verification fail (6/6 as expected). A token minted for one pod cannot be replayed for another; a third party can check "where did my workload run?" without cluster access.

I. E8: B1–B5 overhead and scalability scope

The AKS B1–B5 latency campaign contains two warmups and 30 measured runs per baseline (`performance_b1_b5_high_resolution.csv`); all 150 measured pods succeed, and the anti-quantization check reports a 0.0 ratio of exact 1000 ms multiples. To avoid mixing instruments, Fig. 1 plots only comparable client-observed scheduling-path latencies. The medians are 1170.5 ms for B1 default scheduling, 1189.0 ms for B2 label/nodeSelector, 1168.5 ms for B3 RuntimeClass-only, 1128.0 ms for B4 after the gate is removed, and 1167.0 ms for B5. Separately, B5's instrumented scheduler-internal phase median is 216.232 ms; this value comes from monotonic clock instrumentation inside the attestation scheduler and is not plotted or compared as a B1–B5 end-to-end latency. We therefore claim measured prototype overhead, not a universal Kubernetes latency bound. Scheduler-only scalability (KWOK) and `kind` throughput are retained as CI/debug/regression harnesses; we do not present them as principal cloud security or performance results.

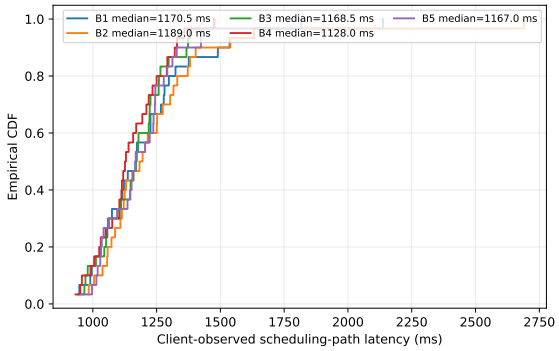

Fig. 1. AKS B1–B5 scheduling-latency CDF using one comparable instrument: high-resolution client-observed scheduling-path timing. B5's separate scheduler-internal phase timing is reported only in text, not mixed into this CDF. Warmups are retained in raw data but excluded from this figure.

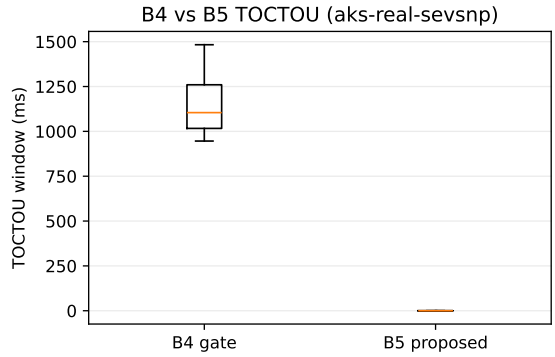

Fig. 2. Central result: the strong scheduling-gate baseline B4 leaves a ∼1.1 s externally controllable gate-removal-to-bind window; our fail-closed PreBind design removes that window from the schedulable path (N=30; $p < 10^{-5}$, Cliff's δ=1.0).

J. E9: Scheduler RBAC minimality audit

The scheduler is part of the TCB, so we audit its own privileges (§X). Twelve `kubectl auth can-i` probes on the real cluster (`scheduler_security_tests.csv`) confirm the nine forbidden capabilities (forge/modify/delete evidence, create raw reports, patch node labels, read arbitrary secrets, create webhook configs, wildcards) are denied and the three required ones allowed. We deliberately scope this as an *RBAC minimality audit*; the complementary hardening (non-root, read-only root filesystem, dropped capabilities, signing key in a Secret not readable via the scheduler's own RBAC, fail-closed on a missing key) is described in §X, and a fuller fake-scheduler / NetworkPolicy / signed-image study is future work.

K. Reproducibility

Within our (private, pending IP review) artifact, the campaign is a single command (`run_full_campaign.sh`): preflight → provision → real-attestation → E1–E9 → statistics/figures → verified teardown. All raw CSVs carry `env`, `run_id`, `seed`, `raw_log_path`; figures come from `analyze_all.py`, `update_q1_final_artifacts.py`, and the AKS performance harness (bootstrap 95% CI, Mann–Whitney U). Public release will follow the IP review.

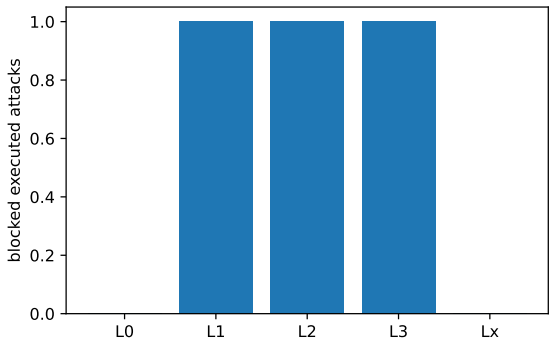

Fig. 3. Ablation on real AKS: with policy only (L0) the forged-label attack bypasses; each added layer (admission, scheduler, token) blocks its attack — the block rate is attributable to the mechanism.

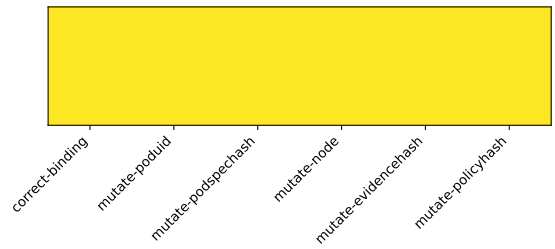

Fig. 4. Identity binding: the correct placement token verifies; mutating any bound field (pod UID, pod-spec/evidence/policy hash, node) is rejected offline.

IX. Security Analysis

We analyse the six properties of §IV against the adversaries ADV-1–ADV-4, and tie each to the attack that would falsify it (Table II) and to a raw result.

a) Placement Safety.: A sensitive pod binds only to a node carrying verified, fresh, non-revoked evidence of the required TEE. ADV-1 presenting revoked (A3) or wrong-TEE (A4) evidence is rejected at Filter; ADV-2/3 forging evidence (A10) is stopped by the trust chain — the node agent cannot write `AttestationEvidence`, and a forged raw report is appraised to `unverified`. Empirically, A4 and A10 were each blocked in 30/30 runs on real AKS.

b) Evidence Freshness.: Stale evidence is rejected at Filter (A2), and evidence that is revoked or expires *during* scheduling is caught by the fail-closed `PreBind` re-check (A7). Because SEV-SNP evidence is necessary but time-bounded — side channels and transient compromise mean a node cannot be trusted forever [9] — freshness and revocation are enforced rather than assumed.

c) Identity Binding.: The placement token binds pod UID, pod-spec hash, node identity, evidence and policy hashes; mutating any bound field makes `verify-placement` fail

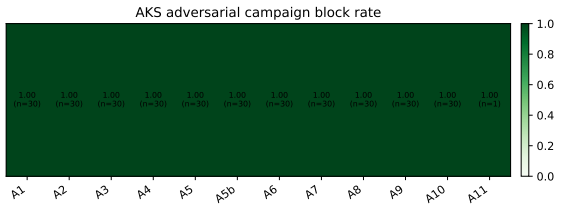

Fig. 5. Deployed adversarial campaign on real SEV-SNP AKS. A1–A10 are each blocked in 30/30 runs; A11 is a single fail-closed GPU-scope control, not a GPU confidential-computing evaluation.

(A8, and the identity-binding experiment). A token minted for one pod cannot be replayed for another.

d) Label Resistance.: Attacker-forgeable metadata cannot grant placement: ADV-4 setting `attested=true` without evidence (A1) leaves the pod `Pending` in all runs, because the scheduler consults cryptographic evidence, not labels.

e) Runtime Consistency.: A production-forbidden simulated runtime on a sensitive pod (A5), a missing model digest (A5b), a policy modified between Filter and PreBind (A6), or a simulated runtime class in production (A9) is refused at admission or at the fail-closed PreBind re-check. The real AKS rerun blocks each Runtime Consistency attack in 30/30 runs: A5 and A5b at admission, A6 via policy-hash mismatch at PreBind, and A9 via production rejection of a labelled simulated `RuntimeClass`.

f) Verifiable Placement.: Every allowed placement carries an Ed25519 token that a third party validates offline. This turns "where did my workload run?" from an assertion into a checkable proof, and composes with scheduler self-security (§X): a compromised node cannot forge evidence and a rogue scheduler cannot forge a verifiable placement.

g) Out of scope.: A malicious API server or etcd, a cloud provider that breaks the SEV-SNP guarantee, microarchitectural side channels, physical attacks and signing-key compromise are outside our model (§IV); they are addressed by the hardware TCB and orthogonal defences, not by orchestration-layer enforcement.

X. Scheduler Self-Security

Because the attestation-aware scheduler becomes a security-critical component — it decides placement and mints the placement token — it must itself be minimally privileged and fail-closed. We therefore treat the scheduler as part of the attack surface and verify, on the live cluster, that its RBAC cannot be abused beyond its role.

A. Minimal RBAC

The scheduler runs under a dedicated `ServiceAccount` whose ClusterRole grants only: *read* on `Pods`, `Nodes`, `Namespaces`, `ConfidentialInferencePolicy` and `AttestationEvidence`; *create/update* on `AIPlacementDecision` (and its status); `create` on `pods/binding` and `bindings` (needed by a second scheduler); `create` on `Events`; and coordination `Lease` access. It has *no* wildcard verbs or resources.

TABLE IV
Scheduler self-security probes (data: `tables/scheduler_security_tests.csv`). "exp" is the required outcome of `auth can-i`.

ID	Capability	exp	Property
S1	create AttestationEvidence	no	no evidence forgery
S2	update AttestationEvidence	no	no evidence tamper
S3	create RawAttestationReport	no	not an attester
S4	update ConfidentialInferencePolicy	no	no policy tamper
S5	patch Nodes	no	no node-label forgery
S6	get Secrets	no	least secret access
S7	get AttestationEvidence	yes	read-only appraisal
S8	create AIPlacementDecision	yes	its output
S9	create pods/binding	yes	second-scheduler bind
S10	delete AttestationEvidence	no	no evidence deletion
S11	create MutatingWebhookConfig	no	no admission hijack
S12	* *	no	no wildcard

B. Fail-closed capability tests

Table IV reports the outcome of capability probes (`kubectl auth can-i -as` the scheduler SA). The scheduler is confirmed *unable* to: create or update `AttestationEvidence` (S1–S2), create `RawAttestationReport` (S3), modify `ConfidentialInferencePolicy` (S4), patch `Node` labels (S5), read arbitrary `Secrets` (S6), delete evidence (S10), create webhook configurations (S11), or use wildcard permissions (S12). It *is* able to read `AttestationEvidence` (S7), create `AIPlacementDecision` (S8), and create `pods/binding` (S9) — exactly its role and nothing more.

C. Signing-key protection and fake-scheduler resistance

The placement-token Ed25519 private key is held in a Kubernetes `Secret` mounted only by the scheduler (or supplied via a sealed env var); the scheduler RBAC does not grant blanket `Secrets` read (S6). Consequently a rogue or "fake" scheduler cannot mint tokens that `verify-placement` accepts: without the signing key it can bind a pod (if it forged RBAC), but the resulting `AIPlacementDecision` would carry no valid token and would fail independent verification. This composes with the trust chain (§IV): a compromised node cannot forge evidence, and a rogue scheduler cannot forge a verifiable placement.

XI. Discussion

a) Why scheduling-time enforcement.: Attestation is often used at secret-release time (CoCo/Trustee) or inside the workload (library OSes). We argue that *placement* is a distinct, earlier control point: deciding *where* a confidential workload runs, and proving it, is orthogonal to and composable with runtime key release. Making verified evidence a scheduling signal turns an implicit assumption ("the operator put me on a good node") into an enforced, checkable property.

b) What the real run demonstrated.: Beyond blocking attacks, the run validated two things that only real hardware can show. First, the end-to-end chain — node agent → MAA JWT → RS256/JWKS verification → real evidence → tokenized placement — works on a production SEV-SNP node. Second, and equally important for a security artifact, the pipeline's *honesty* holds under real conditions: empty and invalid tokens are never promoted to real evidence.

c) Composability and future work.: The design is complementary to pod-level confidential containers: node-level attested placement can front a CoCo runtime. The natural next steps are confidential GPUs (NVIDIA H100 CC) via the GPU-aware policy fields, a self-hosted verifier, millisecond-resolution scheduler instrumentation, and scheduler-only scalability studies. These extend, but do not alter, the node-level guarantees established here.

d) Deferred to later work.: Full key-release orchestration, revocation propagation and audit-chain anchoring are deliberately out of scope for this paper and are the subject of follow-on work; here they appear only insofar as the scheduler consults revocation state.

XII. Threats to Validity

a) Construct validity.: Security is measured as whether the deployed system blocks each attack from an adversarial principal, not as a unit-test assertion; the adversarial campaign runs against the real cluster. The B1–B5 CDF uses high-resolution client-observed timing for all baselines; B5 additionally records scheduler-internal monotonic phase instrumentation, which is reported separately (§XIII).

b) Internal validity.: Each experiment records `env`, `run_id`, `seed` and a `raw_log_path`; warm-ups are retained but excluded from statistics; figures are regenerated from raw CSVs by a single script. B4 is implemented as a genuine strong baseline (real evidence checks, gate removal, default-scheduler bind), not a strawman, so the B4-vs-B5 TOCTOU comparison is fair.

c) External validity.: Results are from one cloud (Azure), one region (westus2), one confidential SKU family (DCasv6/SEV-SNP) and four active confidential nodes in the final AKS multi-node and performance campaign. We do not extrapolate to other TEEs, providers, or larger scales. Local `kind` runs are CI/debug/regression only and are excluded from the main security and performance evidence.

d) Honesty controls.: The verifier refuses to mark empty/invalid MAA tokens real (verified on real hardware and by unit tests); every paper claim maps to a raw file in `claim_evidence_mapping.csv`, and unsupported or out-of-scope claims are labelled as such rather than omitted.

XIII. Limitations

We are explicit about what we do and do not establish.

a) Node-level, not pod-level.: On confidential-VM node pools the attestation evidence refers to the confidential worker node/VM, not to a per-pod TEE. We therefore report *attested-node placement*; we do not claim pod-level workload attestation, which would require a confidential-container runtime (CoCo/Kata-CC) enabled and measured.

b) No confidential GPU, no TDX.: The confidential hardware used is CPU-only AMD SEV-SNP (`DC8as_v6`). We do not evaluate confidential GPUs (e.g. NVIDIA H100 CC) or Intel TDX; GPU-aware placement is expressed in the policy schema but its validation is future work.

c) AI workload scope.: The AKS evaluation includes real governed AI workloads (OpenAI chat, OpenAI embedding, and a local CPU model) to validate attested placement, model/image digest binding, and offline placement verification. These workloads do not establish model confidentiality, OpenAI service-side confidentiality, confidential GPU execution, or large-model throughput.

d) Latency scope.: The AKS timing claim is based on a B1–B5 campaign with 30 measured runs per baseline. The B1–B5 figure uses one comparable client-observed timing instrument; under that instrument, B5's median is 1167.0 ms. B5 also records an in-process scheduler-internal phase metric (median 216.232 ms), but that metric is reported separately and must not be read as an end-to-end speedup over B1–B4. We report prototype overhead, not a universal Kubernetes latency bound.

e) Scale.: The final AKS campaign scaled the `Standard_DC8as_v6` confidential pool to four active nodes under a 32-vCPU DCasv6 quota. This supports multi-node node qualification and B1–B5 overhead measurement, but it is not a cloud-scale throughput study. Local `kind` and KWOK harnesses are retained only for CI/debug/regression or scheduler-only scalability context and are not presented as main cloud security or performance results.

f) MAA as trust anchor.: Real evidence trusts the Microsoft Azure Attestation service and the SEV-SNP hardware within their stated threat model. A verifier that pins MAA signing keys and policy is used; a fully self-hosted verifier is out of scope.

g) Intellectual property.: The placement-token and signed-decision mechanisms are subject to an IP review before submission (marked in the design/implementation sections); the artifact is private pending that review.

XIV. Conclusion

We presented an attestation-aware Kubernetes scheduler that makes verified SEV-SNP attestation a first-class scheduling signal and binds each placement to independently verifiable cryptographic evidence. A trust chain separates the untrusted node attester from a central verifier; a fail-closed `PreBind` re-check removes the externally schedulable gate-removal-to-bind TOCTOU window left by check-then-ungate designs; and an Ed25519 placement token lets a third party verify offline where a workload ran. We formalise six placement properties and argue the architecture enforces them independently of the underlying TEE. On real AMD SEV-SNP confidential

AKS hardware with Azure MAA attestation, the final multi-node snapshot contains four active confidential nodes with verified real evidence, while invalid tokens were never marked real. Our central quantitative result is a head-to-head comparison with a strong scheduling-gate baseline, which leaves a $\sim$1.1 s externally controllable gate-removal-to-bind window ($p < 10^{-5}$, Cliff's δ=1.0) that our design removes from the schedulable path while adding an auditable placement artifact. Layer ablations support the admission/scheduler/token design, the real AKS adversarial campaign blocks A1–A10 in 300/300 instances, the B1–B5 campaign measures 30 runs per baseline with B5 client-observed median 1167.0 ms and a separate scheduler-internal phase median of 216.232 ms, and three governed AI workloads pass placement-token verification. We claim node-level SEV-SNP attested placement only, not pod-level attestation, confidential GPU execution, OpenAI service confidentiality, or model confidentiality. Within our (currently private, pending IP review) artifact, every reported figure is regenerated from raw results by script; public artifact release will follow the IP review.

REFERENCES

[1] M. Misono, D. Stavrakakis, N. Santos, and P. Bhatotia, "Confidential VMs explained: An empirical analysis of AMD SEV-SNP and intel TDX," *Proceedings of the ACM on Measurement and Analysis of Computing Systems (POMACS)*, vol. 8, no. 3, 2024.

[2] P.-C. Cheng, W. Ozga, E. Valdez, S. Ahmed, Z. Gu, H. Jamjoom, H. Franke, and J. Bottomley, "Intel TDX demystified: A top-down approach," *ACM Computing Surveys (also arXiv:2303.15540)*, 2024.

[3] H. Birkholz, D. Thaler, M. Richardson, N. Smith, and W. Pan, "Remote ATtestation procedureS (RATS) architecture," IETF, Tech. Rep. RFC 9334, 2023.

[4] P. Paradžik, A. Derek, and M. Horvat, "Formal security analysis of the AMD SEV-SNP software interface," *arXiv preprint arXiv:2403.10296*, 2024.

[5] G. Chen, Y. Zhang, and T.-H. Lai, "OPERA: Open remote attestation for intel's secure enclaves," in *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2019.

[6] Y. Yang, K. Wang, Y. Luo, H. Yin, J. Cai, S. Zhou, and W. Wang, "Implement Kubernetes pod-level remote attestation for confidential workloads on dstack," *arXiv preprint arXiv:2606.03323*, 2026.

[7] Cloud Native Computing Foundation, "Confidential containers," https://confidentialcontainers.org/, 2024, cNCF sandbox project; accessed 2026-07-04.

[8] Kubernetes Contributors, "Confidential Kubernetes: Use confidential virtual machines and enclaves to improve your cluster security," https://kubernetes.io/blog/2023/07/06/confidential-kubernetes/, 2023, kubernetes Blog.

[9] M. Li, Y. Zhang, H. Wang, K. Li, and Y. Cheng, "CIPHERLEAKS: Breaking constant-time cryptography on AMD SEV via the ciphertext side channel," in *30th USENIX Security Symposium*, 2021.

[10] V. Costan and S. Devadas, "Intel SGX explained," *IACR Cryptology ePrint Archive, Report 2016/086*, 2016.

[11] D. Lee, D. Kohlbrenner, S. Shinde, K. Asanović, and D. Song, "Keystone: An open framework for architecting trusted execution environments," in *Proceedings of the Fifteenth European Conference on Computer Systems (EuroSys)*, 2020.

[12] X. Xu, W. Wang, Y. Wu, C. Wang, H. Zhu, H. Ma, Z. Min, Z. Pang, R. Hou, and Y. Jin, "virtCCA: Virtualized Arm confidential compute architecture with TrustZone," *arXiv preprint arXiv:2306.11011*, 2023.

[13] D. Cerdeira, J. Martins, N. Santos, and S. Pinto, "Rezone: Disarming TrustZone with TEE privilege reduction," *arXiv preprint arXiv:2203.01025 (USENIX Security 2022)*, 2022.

[14] Q. Michaud, S. Ramezanian, D. Ayed, O. Levillain, and J. Garcia-Alfaro, "Abstraction of trusted execution environments as the missing layer for broad confidential computing adoption: A systematization of knowledge," *arXiv preprint arXiv:2512.22090*, 2025.

[15] T. Knauth, M. Steiner, S. Chakrabarti, L. Lei, C. Xing, and M. Vij, "Integrating remote attestation with transport layer security," *arXiv preprint arXiv:1801.05863*, 2018.

[16] K. Akama, Y. Nakatsuka, K. Luke, M. Sato, and K. Uehara, "RA-WEBs: Remote attestation for WEB services," *arXiv preprint arXiv:2411.01340*, 2024.

[17] W. Liu, W. Wang, X. Wang, X. Meng, Y. Lu, H. Chen, X. Wang, Q. Shen, K. Chen, H. Tang, Y. Chen, and L. Xing, "Confidential attestation: Efficient in-enclave verification of privacy policy compliance," *arXiv preprint arXiv:2007.10513*, 2020.

[18] P. Antonino, A. Derek, and W. A. Wołoszyn, "Flexible remote attestation of pre-SNP SEV VMs using SGX enclaves," *arXiv preprint arXiv:2305.09351*, 2023.

[19] S. Arnautov, B. Trach, F. Gregor, T. Knauth, A. Martin, C. Priebe, J. Lind, D. Muthukumaran, D. O'Keeffe, M. L. Stillwell, D. Goltzsche, D. Eyers, R. Kapitza, P. Pietzuch, and C. Fetzer, "SCONE: Secure Linux containers with Intel SGX," in *12th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2016.

[20] C.-C. Tsai, D. E. Porter, and M. Vij, "Graphene-SGX: A practical library OS for unmodified applications on SGX," in *2017 USENIX Annual Technical Conference (ATC)*, 2017.

[21] Y. Shen, H. Tian, Y. Chen, K. Chen, R. Wang, Y. Xu, Y. Xia, and S. Yan, "Occlum: Secure and efficient multitasking inside a single enclave of Intel SGX," in *Proceedings of the 25th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2020.

[22] A. Sarkar and A. Russo, "HasTEE+: Confidential cloud computing and analytics with Haskell," *arXiv preprint arXiv:2401.08901*, 2024.

[23] A. P. P. Hartono, A. Brito, and C. Fetzer, "CRISP: Confidentiality, rollback, and integrity storage protection for confidential cloud-native computing," *arXiv preprint arXiv:2408.06822*, 2024.

[24] W. Liu, H. Chen, S. Huai, Z. Xu, W. Wang, X. Wang, D. Zhang, Z. Li, H. Tang, and Z. Liu, "Characterizing trust boundary vulnerabilities in TEE containers: An empirical study," *arXiv preprint arXiv:2508.20962*, 2025.

[25] F. Tramèr and D. Boneh, "Slalom: Fast, verifiable and private execution of neural networks in trusted hardware," in *International Conference on Learning Representations (ICLR); arXiv:1806.03287*, 2019.

[26] F. Mo, A. S. Shamsabadi, K. Katevas, S. Demetriou, I. Leontiadis, A. Cavallaro, and H. Haddadi, "DarkneTZ: Towards model privacy at the edge using trusted execution environments," in *Proceedings of the 18th International Conference on Mobile Systems, Applications, and Services (MobiSys)*, 2020.

[27] Z. Sun, R. Sun, C. Liu, A. R. Chowdhury, L. Lu, and S. Jha, "ShadowNet: A secure and efficient on-device model inference system for convolutional neural networks," *arXiv preprint arXiv:2011.05905 (IEEE S&P 2023)*, 2020.

[28] D. Li, Z. Zhang, M. Yao, Y. Cai, Y. Guo, and X. Chen, "TEESlice: Protecting sensitive neural network models in trusted execution environments when attackers have pre-trained models," *arXiv preprint arXiv:2411.09945*, 2024.

[29] W. Hua, M. Umar, Z. Zhang, and G. E. Suh, "GuardNN: Secure accelerator architecture for privacy-preserving deep learning," *arXiv preprint arXiv:2008.11632 (DAC 2022)*, 2020.

[30] H. Hashemi, Y. Wang, and M. Annavaram, "DarKnight: An accelerated framework for privacy and integrity preserving deep learning using trusted hardware," *arXiv preprint arXiv:2207.00083 (MICRO 2021)*, 2022.

[31] J. Ma, C. Yu, A. Zhou, B. Wu, X. Wu, X. Chen, X. Chen, L. Wang, and D. Cao, "S3ML: A secure serving system for machine learning inference," *arXiv preprint arXiv:2010.06212*, 2020.

[32] D. Lee, D. Kuvaiskii, A. Vahldiek-Oberwagner, and M. Vij, "Privacy-preserving machine learning in untrusted clouds made simple," *arXiv preprint arXiv:2009.04390*, 2020.

[33] G. Hu, Y. Wu, G. Chen, T. T. A. Dinh, and B. C. Ooi, "SeSeMI: Secure serverless model inference on sensitive data," *arXiv preprint arXiv:2412.11640*, 2024.

[34] R. Schambach, Q. D. Le, S. Arnautov, and C. Fetzer, "EnclaveX: End-to-end confidential AI with CPU/GPU TEEs," *arXiv preprint arXiv:2606.31408*, 2026.

[35] H. Yu, Y. Wang, F. Dai, D. Liu, H. Fan, and X. Gu, "Towards confidential and efficient LLM inference with dual privacy protection," *arXiv preprint arXiv:2509.09091*, 2025.

[36] S. Abdollahi, M. Maheri, S. Siby, M. Kogias, and H. Haddadi, "An early experience with confidential computing architecture for on-device model protection," *arXiv preprint arXiv:2504.08508 (SysTEX 2025)*, 2025.

[37] S. Abdollahi, M. M. Maheri, J. Forough, A. Al Sadi, J. Millar, D. Kotz, M. Kogias, and H. Haddadi, "AgenTEE: Confidential LLM agent execution on edge devices," *arXiv preprint arXiv:2604.18231*, 2026.

[38] F. Mo, Z. Tarkhani, and H. Haddadi, "Machine learning with confidential computing: A systematization of knowledge," *arXiv preprint arXiv:2208.10134 (ACM Computing Surveys)*, 2022.

[39] K. D. Duy, T. Noh, S. Huh, and H. Lee, "Confidential machine learning computation in untrusted environments: A systems security perspective," *arXiv preprint arXiv:2111.03308 (IEEE Access)*, 2021.

[40] Z. Zhang, C. Gong, Y. Cai, Y. Yuan, B. Liu, D. Li, Y. Guo, and X. Chen, "No privacy left outside: On the (in-)security of TEE-shielded DNN partition for on-device ML," *arXiv preprint arXiv:2310.07152 (IEEE S&P 2024)*, 2023.

[41] S. Volos, K. Vaswani, and R. Bruno, "Graviton: Trusted execution environments on GPUs," in *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2018.

[42] T. Hunt, Z. Jia, V. Miller, A. Szekely, Y. Hu, C. J. Rossbach, and E. Witchel, "Telekine: Secure computing with cloud GPUs," in *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2020.

[43] J. Zhu, H. Yin, P. Deng, A. Almeida, and S. Zhou, "Confidential computing on NVIDIA Hopper GPUs: A performance benchmark study," *arXiv preprint arXiv:2409.03992*, 2024.

[44] Z. Gu, E. Valdez, S. Ahmed, J. J. Stephen, M. Le, H. Jamjoom, S. Zhao, and Z. Lin, "Blueprint, bootstrap, and bridge: A security look at NVIDIA GPU confidential computing," *arXiv preprint arXiv:2507.02770*, 2025.

[45] V. Duddu, O. Järvinen, L. J. Gunn, and N. Asokan, "Laminator: Verifiable ML property cards using hardware-assisted attestations," *arXiv preprint arXiv:2406.17548 (ACM CODASPY 2025)*, 2024.

[46] A. Ali, M. Imran, V. Kuznetsov, S. Trigazis, A. Pervaiz, A. Pfeiffer, and M. Mascheroni, "Implementation of new security features in CMSWEB Kubernetes cluster at CERN," *arXiv preprint arXiv:2405.15342*, 2024.

[47] S. Enyedi, "Human-certified module repositories for the AI age," *arXiv preprint arXiv:2603.02512*, 2026.